

Module 1: INTRODUCTION TO IOT

Introduction to IoT: Definition, Characteristics, Physical design of IoT, Things in IOT, IoT Protocols-MQTT, CoAP, HTTP, AMQP, XMPP, Logical Design: functional blocks, communication models, Communication APIs: REST Based, Web Socket Based IoT Enabling Technologies: Introduction to WSN, Cloud computing, Big Data Analytics, Communication Protocols, Embedded Systems
Textbooks:
1. Internet of Things: A Hands-On Approach, by Arshdeep Bahga & Vijay Madisetti, Universities Press, 2014
Reference books:
1. Introduction to IoT, by Sudip Misra, Anandarup Mukherjee, Arijit Roy, Cambridge University Press 2021 2. IoT Fundamentals: Networking Technologies, Protocols, and Use Cases for the Internet of Things – David Hanes, Gonzalo Salgueiro, Patrick Grossetete, Pearson Education, 2017

DEFINITION: A dynamic global network infrastructure with self-configuring capabilities based on standard and interoperable communication protocols where physical and virtual "things" have identities, physical attributes, and virtual personalities and use intelligent interfaces, and are seamlessly integrated into the information network, often communicate data associated with users and their environments.

CHARACTERISTICS OF IOT:

- **Dynamic & Self-Adapting**

IoT devices and systems have the capability to dynamically adapt to changing contexts and take actions based on operational conditions, user context, or sensed environmental parameters.

Example: In a surveillance system with multiple cameras, devices can switch between normal and infrared night vision based on time of day, or adjust from low to high resolution when motion is detected, alerting nearby cameras to do the same.

- **Self-Configuring**

IoT devices can automatically configure themselves, allowing a large number of devices to collaborate with minimal manual intervention. This includes establishing network connections, updating software, and adjusting configurations to suit operational requirements.

Example: Weather monitoring stations automatically connecting to a central server, configuring network parameters, and downloading the latest firmware updates.

- **Interoperable Communication Protocols**

IoT devices typically support standardized, interoperable communication protocols, enabling them to interact seamlessly with devices from different manufacturers as well as with cloud or local infrastructure.

Example: Smart home devices from various brands communicating over protocols like MQTT or CoAP to integrate into a single automation system.

- **Unique Identity**

Each IoT device possesses a unique identifier (such as an IP address, MAC address, or URI) that ensures precise addressing, monitoring, and control. Intelligent interfaces adapt to context, allowing users to query, monitor, and control devices remotely.

Example: A connected thermostat with a unique IP address enabling remote monitoring and control via a mobile app.

- **Integrated into the Information Network**

IoT devices are integrated into broader networks that allow communication, discovery, and exchange of data with other devices and systems. This integration enables collaborative intelligence, where aggregated data leads to smarter decision-making.

Example: A network of weather monitoring nodes exchanging data, which is aggregated and analyzed to generate accurate regional weather forecasts.

- **Connectivity**

Reliable connectivity is the backbone of IoT, enabling devices to exchange data with each other and with cloud platforms in real time. This can involve wired or wireless technologies such as Wi-Fi, Zigbee, LoRa, or cellular networks.

Example: A smart energy meter transmitting usage statistics to the utility company via LTE.

- **Intelligence and Identity**

Many IoT devices embed local intelligence for decision-making and always maintain a unique identity to ensure secure and context-aware interaction.

Example: A factory robot that autonomously adjusts speed and precision based on product type while being uniquely identifiable in the production network.

- **Scalability**

IoT systems must be capable of scaling from a few devices to millions without compromising performance, reliability, or security.

Example: A nationwide network of connected traffic lights that can handle growth in connected intersections as cities expand smart infrastructure.

PHYSICAL DESIGN OF IOT

The **physical design of the Internet of Things (IoT)** refers to the tangible devices and hardware elements that form the building blocks of IoT systems. These components collect data from the environment, process information, communicate over networks, and perform actions. Unlike the conceptual architecture, the physical design emphasizes the actual hardware—sensors, actuators, processors, communication modules, and storage systems—that enable IoT functionality.

IoT devices vary in complexity, ranging from simple microcontroller-based systems to sophisticated microprocessor-driven platforms integrated with cloud infrastructure. The following sections highlight the **Physical Devices (building blocks)** of IoT.

a) Microcontrollers

- Microcontrollers are compact computing units optimized for specific IoT tasks. They typically integrate CPU, memory, and input/output (I/O) peripherals into a single chip.
- Features:
 - Low power consumption (suitable for battery-operated devices).
 - On-chip Flash and SRAM.
 - GPIO (General Purpose Input/Output) pins for direct connection with sensors and actuators.
 - Cost-effective and reliable for small-scale IoT deployments.
- Examples:
 - *Arduino UNO/Mega* → Widely used for prototyping IoT projects.
 - *ESP8266 / ESP32* → Microcontrollers with built-in Wi-Fi, ideal for IoT connectivity.
 - *Raspberry Pi Pico* → Low-cost, ARM Cortex-M0+ microcontroller suitable for learning and low-power IoT tasks.
- Use Case: A smart irrigation system using Arduino and ESP8266 to collect soil moisture data and transmit it wirelessly.

b) Microprocessors

- Microprocessors are more powerful than microcontrollers. They act as central computing units, capable of running operating systems like Linux, handling multitasking, and supporting high-speed computation.
- Features:
 - High processing speed (GHz range).
 - Large memory support and external peripherals.
 - Can run complex IoT applications and analytics locally (edge computing).
- Examples:
 - *ARM Cortex Processors* → Found in smartphones and advanced IoT boards.
 - *Intel Atom* → Designed for embedded IoT applications with higher computational requirements.
 - *Raspberry Pi (Model 3/4)* → Low-cost processor board with Linux OS support, making it suitable for gateways and IoT servers.
- Use Case: Smart home hub using Raspberry Pi 4 as a central processor to manage multiple IoT devices.

c) Sensors

- Sensors are devices that sense or measure physical conditions from the surrounding environment and convert them into electrical signals.
- Types of Sensors & Examples:
 - *Temperature Sensors*: LM35, DHT11/DHT22.
 - *Humidity Sensors*: DHT11, hygrometers.
 - *Motion Sensors*: Passive Infrared (PIR).
 - *Light Sensors*: LDR, photodiodes.
 - *Gas Sensors*: MQ series (CO2, LPG, methane detection).
- Role in IoT: They are the primary data sources, enabling IoT systems to monitor environments and trigger automated actions.
- Use Case: DHT22 sensor monitoring temperature and humidity in a greenhouse and sending data to the cloud.

d) Actuators

- Actuators are devices that take input signals (electrical) and produce physical actions. They allow IoT systems to interact with the real world.
- Examples:

- Motors (DC, servo, stepper) → Robotics and automation.
- Relays → Switching appliances remotely.
- LEDs and buzzers → Visual and audio alert systems.
- Importance: They transform IoT devices from passive observers into active controllers of their environments.
- Use Case: Automatic door lock system controlled via IoT-enabled actuators.

e) Communication Modules

- These modules provide connectivity and enable IoT devices to exchange data with each other and with the cloud.
- Examples:
 - Wi-Fi modules (ESP8266, ESP32) → Short-range IoT.
 - Bluetooth modules (HC-05, HC-06) → Wearables and smart gadgets.
 - LoRa modules → Long-range IoT networks.
 - GSM/4G/5G modules (SIM800L, SIM900) → Cellular IoT applications.
- Role: Ensure real-time, reliable, and scalable communication between IoT devices.
- Use Case: Smart water meter using LoRa to transmit usage data over long distances.

f) Gateways and Routers

- Gateways serve as intermediaries between local IoT devices and the broader Internet. They manage device connectivity, security, and protocol translation.
- Functions:
 - Protocol conversion (e.g., Zigbee → IP).
 - Data buffering and filtering.
 - Security enforcement.
- Examples: Cisco IoT gateways, Intel IoT Gateways, Raspberry Pi-based custom gateways.
- Use Case: Smart city streetlights connected via Zigbee, with a gateway converting data to cloud-based IoT platforms.

g) Cloud and Storage Units

- IoT generates large volumes of data that require secure storage, processing, and analytics. Cloud platforms provide scalable and distributed infrastructure for IoT.
- Examples:
 - AWS IoT Core.
 - Microsoft Azure IoT Hub.

- Google Cloud IoT Core.
- Ubidots (IoT visualization platform).
- Role:
 - Long-term data storage.
 - Big data analytics.
 - Machine learning integration for predictions.
 - Remote device management.
- Use Case: Smart healthcare wearable devices storing patient health records securely on AWS IoT Cloud.

1. THINGS IN IOT

The “Things” in IoT refer to IoT devices that possess unique identities and capabilities for remote sensing, actuating, and monitoring. These devices can:

- Exchange data with other connected devices and applications (directly or indirectly).
- Collect data from other devices and process it locally or send it to centralized servers or cloud-based application back-ends.
- Perform certain tasks locally and others within the IoT infrastructure depending on memory, processing power, communication latency, network speeds, and deadlines.

A typical IoT device consists of several interfaces:

1. I/O interfaces for connecting sensors and actuators.
2. Internet connectivity interfaces (wired and wireless).
3. Memory and storage interfaces for data handling.
4. Audio/Video interfaces for multimedia communication.

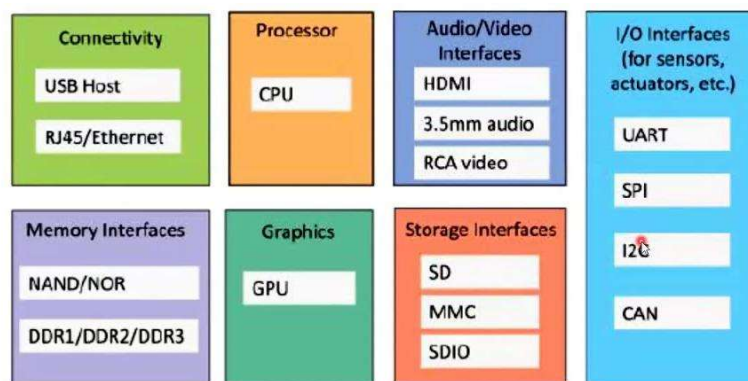


Figure 1.1: Generic Block Diagram of an IoT Device

IoT devices can:

- Collect data from onboard or attached sensors (e.g., temperature, humidity, light intensity).
- Communicate sensed data to other devices or cloud servers.
- Control actuators to interact with physical systems (e.g., turn appliances on/off).

Example:

A relay switch connected to an IoT device can automatically switch an appliance on or off based on commands received over the Internet.

IoT devices come in various forms:

- Wearable sensors
- Smartwatches
- LED lights
- Automobiles
- Industrial machines



Figure 1.2: IoT Devices

Almost all IoT devices generate data, which when processed by analytics systems, provides actionable insights. For instance, soil moisture monitoring devices can collect data that helps determine optimal watering schedules.

2. IOT PROTOCOLS

In the Internet of Things (IoT) ecosystem, protocols define the rules and formats for communication between devices, gateways, servers, and applications. They determine how data is structured, transmitted, and received across the IoT network. IoT protocols are designed to operate efficiently over constrained environments, where devices may have limited computing resources, low bandwidth, and intermittent connectivity.

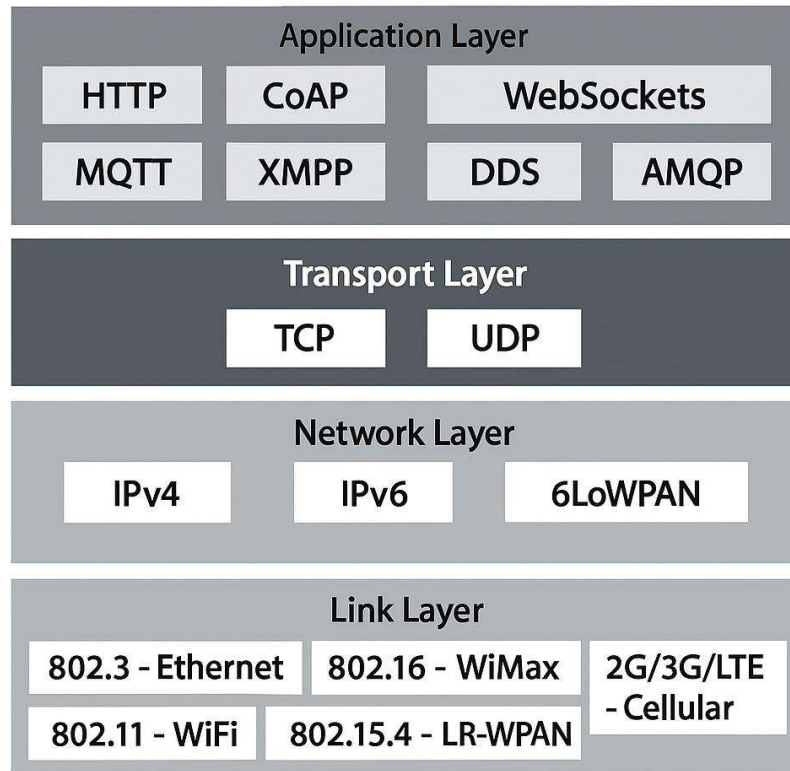


Figure 2.1: IoT Protocols

The Application Layer in the IoT protocol stack provides the interface between IoT devices and end-user applications.

It defines how data is formatted, encoded, transmitted, and interpreted, ensuring interoperability between devices, services, and platforms.

At this layer, protocols are application-specific but must be lightweight, scalable, and optimized for constrained IoT devices.

1. MQTT (Message Queue Telemetry Transport)

- MQTT is a **lightweight pub/sub messaging protocol** for IoT. Devices (clients) connect to a central **broker**. Publishers send messages to *topics* on the broker, and the broker forwards each message to all clients subscribed to that topic. This decouples publishers and subscribers (they need not know each other).
- **Features:** Very low protocol overhead (MQTT headers can be as small as 2 bytes). Operates over TCP with optional QoS levels (for reliable delivery). Brokers support retained messages, last-will notifications, and efficient many-to-many distribution. Designed for intermittent or low-bandwidth links, MQTT is *well-suited for constrained environments*.
- **Example:** In a smart home, a temperature sensor publishes its readings every minute to topic `home/groundfloor/temperature`. A thermostat controller and a smartphone app are subscribed to that topic, so they immediately receive the latest temperature. This lets the controller adjust heating in real time and the user monitor conditions.

MQTT Publish/Subscribe Framework

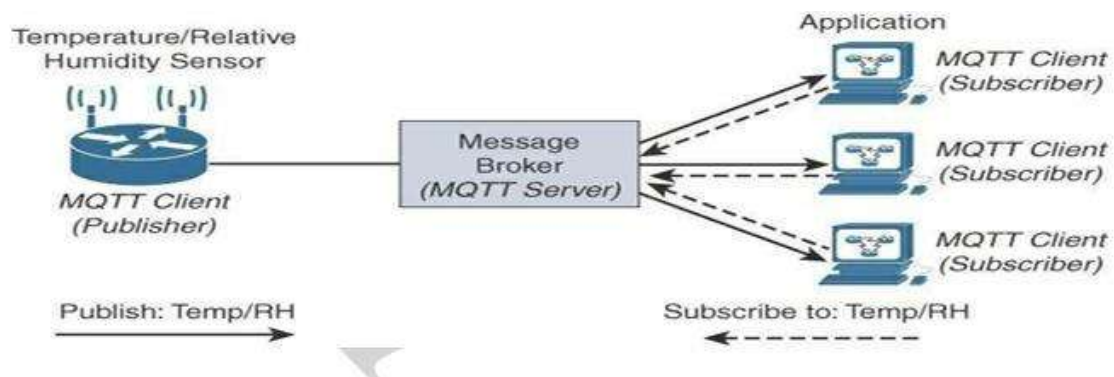


Figure 2.2 MQTT Publish/Subscribe Framework

An **MQTT client** can act as a **publisher** to send data (or resource information) to an **MQTT server** acting as an **MQTT message broker**.

- In the figure, the MQTT client on the **left side** is a **Temperature (Temp)** and **Relative Humidity (RH)** sensor that publishes its Temp/RH data.
- The **MQTT server (message broker)** accepts the network connection along with application messages, such as Temp/RH data, from the publishers.

- It also handles the **subscription** and **unsubscription** process and **pushes** the application data to MQTT clients acting as subscribers.
- The application on the **right side** of the figure is an MQTT client that is a **subscriber** to the Temp/RH data being generated by the publisher or sensor on the left.
- This model, where **subscribers express a desire to receive information from publishers**, is widely used in publish/subscribe systems.

The presence of a **message broker** in MQTT **decouples** the data transmission between clients acting as publishers and subscribers.

- Publishers and subscribers **do not need to know about each other's existence**.
- A key benefit of this decoupling is that the MQTT message broker can **buffer and cache** information in case of network failures, ensuring reliable delivery when the network is restored.

MQTT Message Format

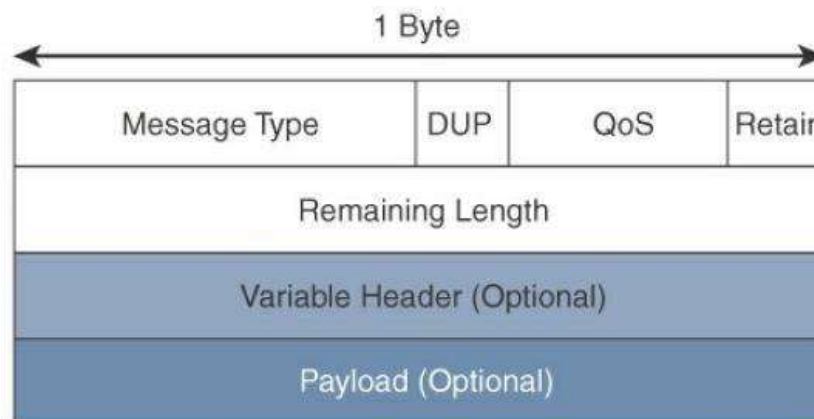


Figure 2.3 MQTT Message Format

MQTT messages are exchanged between clients and the broker in a specific control packet structure, which ensures efficient, lightweight communication. The MQTT message consists of three main parts:

1. Fixed Header (Present in all MQTT Control Packets)

The Fixed Header contains mandatory fields and defines the type of MQTT control message and various control flags.

- Message Type – Indicates the type of MQTT control packet (e.g., CONNECT, PUBLISH, SUBSCRIBE, etc.).

- DUP (Duplicate Delivery Flag) – Set to 1 if the message is a duplicate of an earlier attempt (for retransmissions).
- QoS (Quality of Service Level) – Indicates the delivery guarantee level:
 - 0 – At most once (no acknowledgment).
 - 1 – At least once (requires acknowledgment).
 - 2 – Exactly once (two-phase handshake).
- Retain – When set, instructs the broker to retain the last message for new subscribers.
- Remaining Length – Specifies the number of bytes remaining in the message (variable header + payload).

2. Variable Header (Optional)

The Variable Header is present in some control packets and contains additional control information specific to the message type.

Examples:

- For CONNECT – Contains protocol name, version, connect flags, and keep-alive timer.
- For PUBLISH – Contains the topic name and packet identifier (if QoS > 0).

3. Payload (Optional)

The Payload contains the actual application data being transmitted.

Examples:

- For PUBLISH – Contains the actual sensor readings, status updates, or messages.
- For SUBSCRIBE – Contains the list of topic filters to which the client wants to subscribe.
- For CONNECT – Contains the client identifier, username, and password (if used).

MQTT Control Field

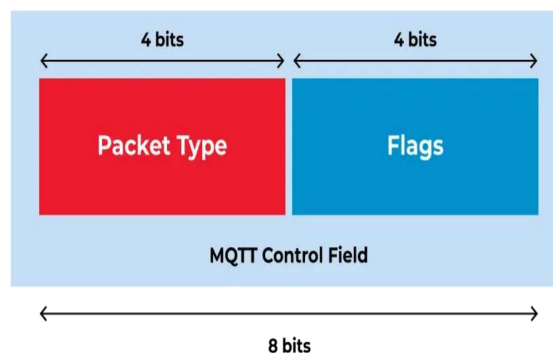


Figure 2.4 MQTT Control Field

The figure shows the MQTT Control Field, which is a part of the fixed header in every MQTT Control Packet. It is always 1 byte (8 bits) and is divided into two parts:

1. Packet Type (4 bits)

- Represents the type of MQTT Control Packet.
- Since 4 bits are used, it can represent 16 different packet types (0–15).
- Examples:
 - 0001 → CONNECT
 - 0010 → CONNACK
 - 0011 → PUBLISH
 - 0100 → PUBACK
 - 1000 → SUBSCRIBE
 - 1001 → SUBACK
 - 1110 → DISCONNECT

Thus, the packet type tells whether the control packet is for connection establishment, publishing messages, acknowledgments, subscription handling, or disconnecting.

2. Flags (4 bits)

- Provides additional control information depending on the packet type.
- Examples:
 - For PUBLISH packets, the flags specify:
 - DUP → Duplicate delivery (whether the message is resent).
 - QoS → Quality of Service level (0, 1, or 2).
 - RETAIN → Whether the broker should store the last message.
 - For other packets, flags may be reserved and must be set to specific values (like 0000).

MQTT Message Types

The following table lists the MQTT protocol messages exchanged between a client and a broker:

MQTT message	Description with direction of flow
CONNECT	Client to server (i.e., Broker), the client requests to connect to the server.
CONNACK	Server to client, connect acknowledgment
PUBLISH	Client to server OR server to client Publish message
PUBACK	Client to server OR server to client Publish acknowledgment
PUBREC	Client to server OR server to client Publish received
PUBREL	Client to server OR server to client Publish release
PUBCOMP	Client to server OR server to client Publish complete
SUBSCRIBE	Client to server Client subscribe request
SUBACK	Server to client Subscribe acknowledgment
UNSUBSCRIBE	Client to server Unsubscribe request
UNSUBACK	Server to client Unsubscribe acknowledgment
PINGREQ	Client to server Ping request
PINGRESP	Server to client PING RESPONSE
DISCONNECT	Client to server, Client is disconnecting

Applications of MQTT

1. Smart Homes and Building Automation

- Used in smart lighting, HVAC (Heating, Ventilation, Air Conditioning), and energy management.
- Example: A smart thermostat publishes temperature data, and smart appliances subscribe to it for automated control.

2. Industrial IoT (IIoT) and Manufacturing

- Real-time monitoring of machines and predictive maintenance.
- MQTT ensures reliable message delivery even with intermittent connectivity in factories.

3. Healthcare and Wearables

- IoT-enabled medical devices send patient data (e.g., heart rate, blood pressure) to hospital servers or doctors.
- Useful in remote patient monitoring and telemedicine.

4. Automotive and Transportation

- Vehicle telematics (GPS, fuel usage, diagnostics) uses MQTT to transmit data to fleet management systems.
- Supports real-time vehicle tracking and predictive maintenance.

5. Agriculture and Smart Farming

- Sensors publish soil moisture, temperature, and humidity data.
- Automated irrigation systems subscribe and take action accordingly.

6. Energy Systems and Smart Grids

- Used for monitoring power consumption, solar panels, and wind turbines.
- Enables demand-response management in smart grids.

7. Supply Chain and Logistics

- Shipment tracking sensors publish location and environmental conditions (e.g., temperature of perishable goods).
- Logistics platforms subscribe for real-time updates.

8. Surveillance and Security Systems

- Security cameras and motion sensors publish alerts.
- Monitoring applications subscribe to trigger alarms or notifications.

9. Social Media and Messaging Apps

- Some real-time chat and collaboration applications (similar to Twitter-like updates) use MQTT for instant delivery of notifications.

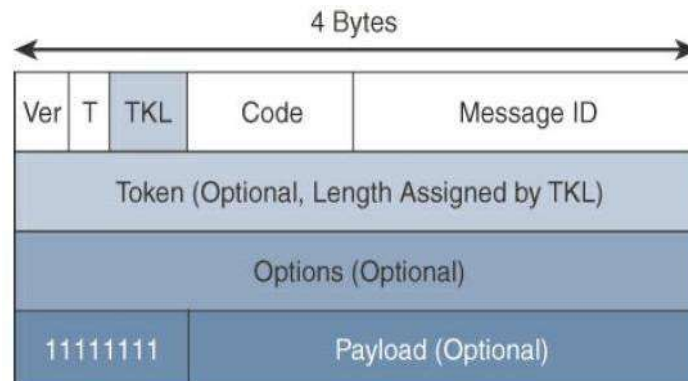
10. Environmental Monitoring

- Deployed in smart cities for monitoring air quality, pollution levels, and weather data.
- Data is published from distributed sensors and subscribed by analytics dashboards.

2. CoAP (Constrained Application Protocol)

- CoAP is a **web-transfer protocol for constrained devices** (often called “HTTP for IoT”). Like HTTP, it uses a RESTful request/response model with methods such as GET, PUT, POST, DELETE. However, CoAP runs over UDP (not TCP), uses much smaller headers, and adds IoT-specific features. It was designed by the IETF CoRE group to easily translate to HTTP for interoperability.
- **Features:** Client–server architecture with *methods* similar to HTTP. Messages are exchanged in UDP datagrams, so CoAP is lighter and can do simple multicast (useful for one-to-many updates). CoAP includes an “observe” extension that lets a client subscribe to resource changes (a simple form of pub/sub). Overall CoAP **minimizes header overhead** and complexity, making it ideal for simple sensors/actuators on 8-bit microcontrollers.
- **Example:** A network of battery-powered environmental sensors (temperature, humidity) might expose readings via CoAP. A controller can GET the current values or subscribe (using CoAP Observe) to be notified whenever they change. Similarly, a mobile app could send a CoAP PUT to turn on a smart light by addressing the light’s IP and resource path.

CoAP Message Format

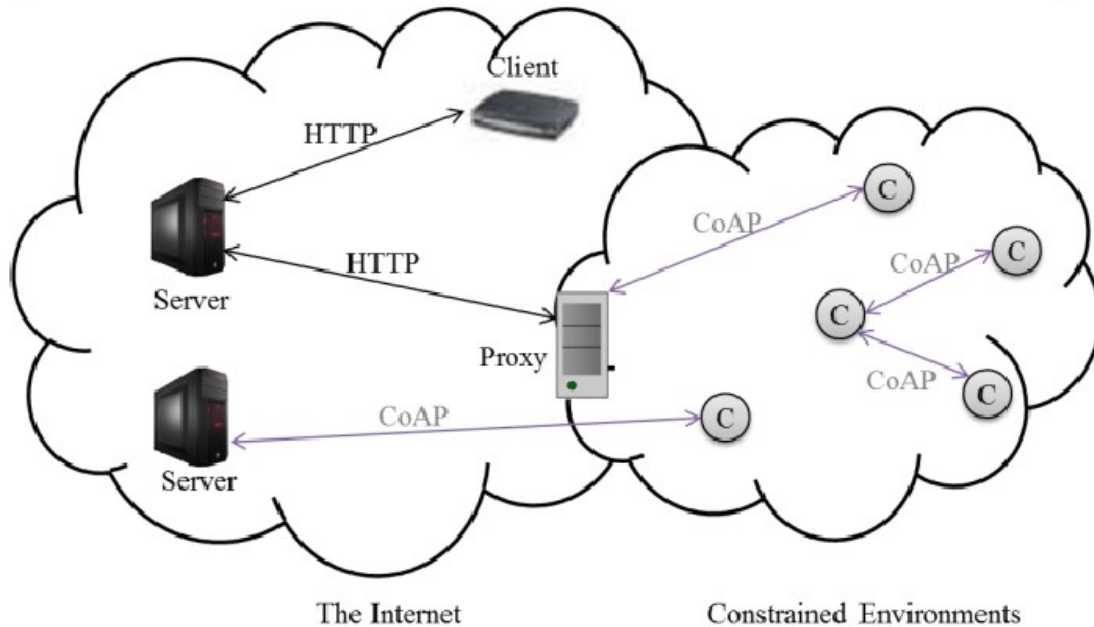


CoAP messages are encoded in **binary-format** or 0/1 format. Like other message formats, CoAP message has a header and a payload section along with an optional section. The size of CoAP header is **4 bytes** or 32 bits. This size is fixed for every CoAP message. Whereas the other part of message is the optional part which includes payload and tokens of variable size ranging from **0-8 bytes**. The message format of CoAP contains the following fields:

- **Version** - The size of version field is 2 bits. It represents the version of the CoAP protocol.
- **Type Code** - The size of type field is 2 bits. There are four types of messages namely confirmable, non-confirmable, acknowledgement and reset represented by the bit patterns 00, 01, 10, 11 respectively.
- **Option Count** - The size of option count field is 4 bits. These 4 bits, means there could be a total of 16 possible options in header.
- **Code** - The size of code field is 8 bits. This indicates whether message is empty, request message or response message.
- **Message ID** - The size of message ID field is 16 bits. It is used to detect the message duplication and types of messages.
- **Tokens** [Optional] - The size of tokens field is variable which ranges from 0-8 bytes. It's used to match a response with request.
- **Options** [Optional] - The options field in CoAP message has a variable size. It defines the type of payload message.
- **Payload** [Optional] - Similar to options field, the payload field has a variable size. The payload of requests or of responses is typically a representation of the requested resource or the result of the requested action.

CoAP Architecture

The **Constrained Application Protocol (CoAP)** is designed to enable communication between **constrained devices** (like sensors, actuators, embedded systems) in IoT and applications on the Internet. The architecture bridges **constrained environments** with the broader **Internet** using **CoAP and HTTP**.



Components in the Architecture

1. CoAP Clients (C)

- These are IoT devices such as temperature sensors, smart meters, or light controllers.
- They use the CoAP protocol to send requests (GET, POST, PUT, DELETE) and receive responses.
- CoAP clients are resource-constrained and communicate via **UDP** for efficiency.

2. CoAP Servers

- IoT devices can also act as servers by hosting **resources**.
- Example: A sensor acting as a server provides its readings when a client requests them.
- CoAP servers respond with lightweight data packets optimized for low-power networks.

3. Proxy

- The **Proxy** acts as a **translator or bridge** between the CoAP world (constrained networks) and the traditional Internet (HTTP).
- Example: A CoAP sensor in a smart home can send its data through a CoAP-HTTP proxy to an Internet-based cloud application.

4. HTTP Clients and Servers (Internet Side)

- On the Internet side, **standard clients (e.g., browsers, apps)** use HTTP.
- These interact with **servers (e.g., cloud platforms like AWS IoT, Azure IoT Hub, or web servers)**.
- The proxy converts CoAP messages to HTTP (and vice versa) so that constrained devices can communicate seamlessly with Internet applications.

Working of CoAP Architecture

1. Within Constrained Environments

- Devices communicate directly using **CoAP over UDP**.
- Example: A sensor (CoAP server) responds to a CoAP client request for data.

2. Crossing the Boundary to the Internet

- A **CoAP-HTTP Proxy** translates between CoAP (for IoT devices) and HTTP (for Internet applications).
- Example: A mobile app using HTTP can still access IoT devices that speak CoAP through the proxy.

3. On the Internet Side

- Communication continues using standard **HTTP protocol**.
- Data from IoT devices is sent to Internet servers for **processing, storage, or visualization**.

Example Scenario

- In a **smart home**, temperature sensors and light controllers communicate via CoAP.
- A **CoAP Proxy** translates these CoAP messages into HTTP.

- A **cloud application** (on the Internet) receives the data in HTTP format, processes it, and allows users to control devices remotely.

Applications of CoAP

- **Real Time Monitoring in Grid** - Smart cities can monitor the distribution and generation of power remotely. The CoAP sensors could be embedded inside the transformers and the data could be transferred over GPRS or 6LowPAN.
- **Defense utilities** - The armory and tanks are now-a-days fitted with sensors so that information could be communicated remotely without any interference. The CoAP sensors could detect any intrusion. This makes them capable to transfer more data even under low bandwidth network.
- **Aircraft utilities** - The Aircraft sensors and actuators could be connected with other sensors and communication can take place using smart CoAP based sensors and actuators.

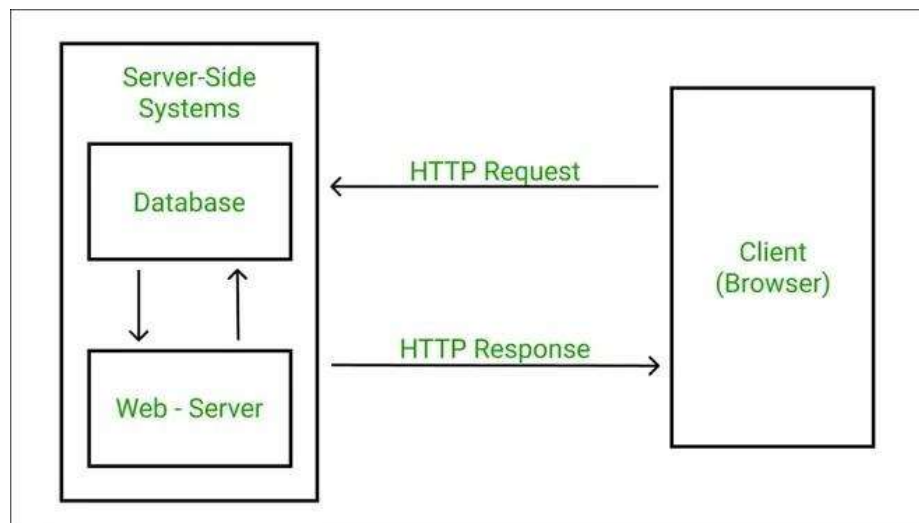
3. HTTP (Hypertext Transfer Protocol)

- HTTP is the standard Web application protocol over TCP. It is **stateless and request/response-based**. Clients send HTTP requests (GET, POST, PUT, DELETE, etc.) to a server; the server responds with data (HTML, JSON, etc.). Because it is so ubiquitous, HTTP is often used in IoT when devices or gateways are powerful enough.
- **Features:** Relies on TCP (connection-oriented), so it has higher overhead (reliability, congestion control). Each request is independent (stateless). HTTP/REST in IoT typically uses JSON or XML payloads. It's easy to integrate with web services and cloud APIs, but its verbosity makes it less ideal for tiny sensors.
- **Example:** A smart energy meter posts its usage data once an hour to a cloud server via an HTTP POST with a JSON payload. A cloud-hosted dashboard pulls data via HTTP GET to display meter readings. Also, many IoT devices (cameras, routers) host simple HTTP servers to allow configuration through a web browser.

HTTP Request/Response:

HTTP is a request-response protocol, which means that for every request sent by a client (typically a web browser), the server responds with a corresponding response. The basic flow of an HTTP request-response cycle is as follows:

- **Client sends an HTTP request:** The client (usually a web browser) initiates the process by sending an HTTP request to the server. This request includes a request method (GET, POST, PUT, DELETE, etc.), the target URI (Uniform Resource Identifier, e.g., a URL), headers, and an optional request body.
- **Server processes the request:** The server receives the request and processes it based on the requested method and resource. This may involve retrieving data from a database, executing server-side scripts, or performing other operations.
- **Server sends an HTTP response:** After processing the request, the server sends an HTTP response back to the client. The response includes a status code (e.g., 200 OK, 404 Not Found), response headers, and an optional response body containing the requested data or content.
- **Client processes the response:** The client receives the server's response and processes it accordingly. For example, if the response contains an HTML page, the browser will render and display it. If it's an image or other media file, the browser will display or handle it appropriately.



Methods of HTTP

- **GET:** Used to retrieve data from a specified resource. It should have no side effects and is commonly used for fetching web pages, images, etc.
- **POST:** Used to submit data to be processed by a specified resource. It is suitable for form submissions, file uploads, and creating new resources.

- **PUT:** Used to update or create a resource on the server. It replaces the entire resource with the data provided in the request body.
- **PATCH:** Similar to PUT but used for partial modifications to a resource. It updates specific fields of a resource rather than replacing the entire resource.
- **DELETE:** Used to remove a specified resource from the server.
- **HEAD:** Similar to GET but retrieves only the response headers, useful for checking resource properties without transferring the full content.
- **OPTIONS:** Used to retrieve the communication options available for a resource, including supported methods and headers.
- **TRACE:** Used for debugging purposes to echo the received request back to the client, though it's rarely used due to security concerns.
- **CONNECT:** Used to establish a tunnel to the server through an HTTP proxy, commonly used for SSL/TLS connections.

HTTP Packet Format

HTTP Packet Format

HTTP Request	Request Line
	Header Fields
	Blank Line
	Message Body
HTTP Response	Status Line
	Header Fields
	Blank Line

The **Hypertext Transfer Protocol (HTTP)** is an application layer protocol used for communication between clients (usually web browsers, IoT devices, or applications) and servers (web servers, APIs, or cloud services). Every HTTP message (also referred to as an HTTP packet) has a **well-defined format** that ensures correct communication between client and server.

1. Start-Line

The first line in every HTTP message, known as the **start-line**, indicates the **nature of the message**.

For Requests:

- **<Method> <Request-Target> <HTTP-Version>**
 - **Method:** Specifies the action to be performed. Common methods include:
 - GET → Retrieve data.
 - POST → Send data to the server.
 - PUT → Update a resource.
 - DELETE → Remove a resource.
 - **Request-Target:** The resource being requested (e.g., /index.html).
 - **HTTP-Version:** Protocol version (e.g., HTTP/1.1 or HTTP/2.0).

Example:

GET /products/item1.html HTTP/1.1

For Responses:

- **<HTTP-Version> <Status-Code> <Reason-Phrase>**
 - **HTTP-Version:** Indicates which version the response follows.
 - **Status-Code:** Numeric code indicating the result of the request.
 - 200 → OK (successful).
 - 404 → Not Found.
 - 500 → Server Error.
 - **Reason-Phrase:** Human-readable description of the status.

Example:

HTTP/1.1 200 OK

2. Header Fields

Following the start-line, **headers** provide metadata about the request or response. They are written in **key-value pairs**:

Header-Name: Header-Value

Types of headers:

- **General Headers:** Common to both request and response (e.g., Date, Connection).
- **Request Headers:** Give information about the client or expected response (e.g., Host, User-Agent, Accept).
- **Response Headers:** Provide server details or instructions (e.g., Server, Set-Cookie).
- **Entity Headers:** Describe the body content (e.g., Content-Type, Content-Length).

Examples:

Host: www.example.com

User-Agent: Mozilla/5.0

Content-Type: application/json

Content-Length: 256

Headers **instruct the receiver** on how to interpret and process the data.

3. Blank Line (CRLF)

- After headers, there is always a **blank line** (\r\n).
- This line acts as a **separator** to mark the end of headers and the beginning of the body.
- Without this blank line, the server cannot distinguish headers from the actual content.

4. Message Body

- The body is **optional**.
- It carries the **payload** (actual data being transmitted).
- In **GET requests**, the body is usually absent.
- In **POST/PUT requests**, the body contains data like:
 - HTML form data.
 - JSON objects.
 - XML payloads.
 - Binary data (files, images, videos).

The **Content-Length** header defines the body size, while **Content-Type** describes the format.

Example of HTTP Communication

Request Packet (from Client to Server)

POST /api/data HTTP/1.1

Host: www.iotserver.com

User-Agent: IoT-Device/1.0

Content-Type: application/json

Content-Length: 47

```
{"temperature": 29.5, "humidity": 68, "device": "A1"}
```

- Start-Line → POST /api/data HTTP/1.1
- Headers → Host, User-Agent, Content-Type, Content-Length
- Blank Line → separates headers and body
- Body → JSON data from IoT sensor

Response Packet (from Server to Client)

HTTP/1.1 200 OK

Date: Tue, 19 Aug 2025 10:45:00 GMT

Server: Apache/2.4

Content-Type: application/json

Content-Length: 36

```
{"status": "success", "message": "Data saved"}
```

- Start-Line → HTTP/1.1 200 OK
- Headers → Date, Server, Content-Type, Content-Length
- Blank Line → separator
- Body → JSON confirmation message

APPLICATIONS

1. Web Browsing

- HTTP is the foundation of data communication on the **World Wide Web (WWW)**.
- Used to transfer web pages (HTML, CSS, JavaScript) from the server to the client browser.
- Example: Accessing websites like Google, Wikipedia, Amazon.

2. Data Exchange in Web Services

- Widely used in **RESTful APIs** to exchange data between client and server.
- Common in applications such as mobile apps, IoT devices, and cloud services.

- Example: Weather apps fetching live data from servers via HTTP requests.

3. File Transfer

- HTTP allows downloading and uploading of files over the internet.
- Example: Downloading PDFs, images, or software from a server.

4. Streaming Media

- HTTP is used in adaptive streaming protocols such as **HLS (HTTP Live Streaming)** and **MPEG-DASH**.
- Enables video and audio streaming platforms.
- Example: YouTube, Netflix, Spotify.

5. E-commerce Applications

- Facilitates secure transactions and dynamic page loading in online shopping websites.
- Combined with HTTPS (secure HTTP) for payment gateways.
- Example: Amazon, Flipkart.

6. IoT Applications

- Lightweight IoT devices often use **HTTP** for device-to-cloud communication.
- Example: Smart home devices sending sensor data to cloud servers.

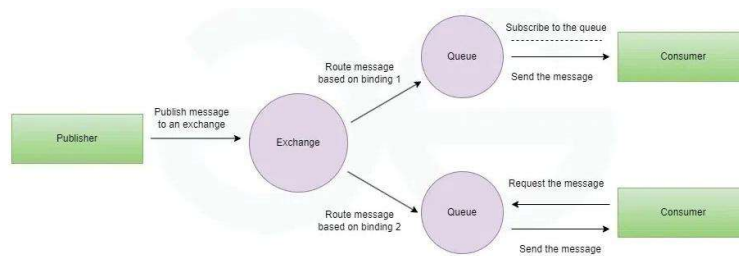
7. Email and Chat Applications

- Some email clients and messaging platforms rely on **HTTP-based APIs** for retrieving and sending messages.
- Example: Web-based email clients (Gmail, Outlook Web) and apps like Slack.

4. AMQP (Advanced Message Queuing Protocol)

- It is an **open application-layer messaging protocol** (originated for business messaging). It standardizes message formats and broker behaviors for reliable, enterprise-grade queues and topics. AMQP operates over TCP with a broker (e.g. RabbitMQ, Apache Qpid).

- Features:** Supports both **point-to-point queues** and **publish/subscribe (via exchanges/topics)**. Messages published to an *exchange* can be routed to one or more queues based on rules. AMQP provides advanced features like guaranteed delivery, transactions, message acknowledgments, and flexible routing. It is heavier than MQTT but offers strong reliability and security.
- Example:** In an industrial IoT deployment, sensors on the factory floor send telemetry messages to an AMQP exchange on a central server. One queue bound to that exchange might deliver data to a monitoring dashboard, while another queue archives the data. Because AMQP queues can persist messages, temporary disconnections do not result in data loss – the broker holds messages until consumers retrieve them.



AMQP is an acronym used for the Advanced Message Queuing Protocol. It is a protocol that is used for communication between applications. It is a lightweight, protocol that supports the applications for data transfer. This protocol is used for its scalability and modularity with the technologies.

Components of AMQP

- Exchanges:** The **exchange** is responsible for fetching messages and properly arranging them in the appropriate queue
- Channel:** A channel is a multiplexed virtual connection between AMQP peers that is built into an existing connection.
- Message Queue:** It is a unique entity that connects messages to their resources or points.
- Binding:** Bindings are a set of predetermined instructions for queuing and exchanging. It manages message transmission and delivery.

- **Virtual Host:** Vhost is a platform that provides isolation capabilities within the broker. Multiple vhosts may be functional at the same time, depending on the users and their access rights.

Applications

1. Industrial IoT Systems

- Used for factory automation where thousands of machines and sensors generate telemetry data.
- Example: Sensor nodes publish data to a central RabbitMQ broker, which routes it to monitoring dashboards, databases, and control systems.
- Ensures no data loss even if the network is unstable (messages persist in queues).

2. Financial Services

- Stock exchanges, banking systems, and payment gateways use AMQP for real-time transaction messaging.
- Guarantees transaction integrity, ordering, and reliability, which are critical in financial operations.

3. Cloud Messaging and Microservices

- AMQP underpins many cloud platforms (e.g., Microsoft Azure Service Bus, OpenStack).
- Microservices communicate via AMQP brokers, enabling scalable, asynchronous workflows.

4. Smart Cities and Utilities

- Used in smart grid management, water monitoring, and transportation systems.
- AMQP allows diverse devices and systems to exchange event-based data with guaranteed delivery.

5. Enterprise Applications

- ERP, CRM, and supply chain systems rely on AMQP for message-based integration.
- Supports publish/subscribe and point-to-point queues, ensuring flexible workflows.

6. Healthcare Systems

- Medical IoT devices (e.g., patient monitoring sensors) send critical health data via AMQP.
- Data is reliably queued and delivered to hospital dashboards and cloud storage, ensuring availability for doctors.

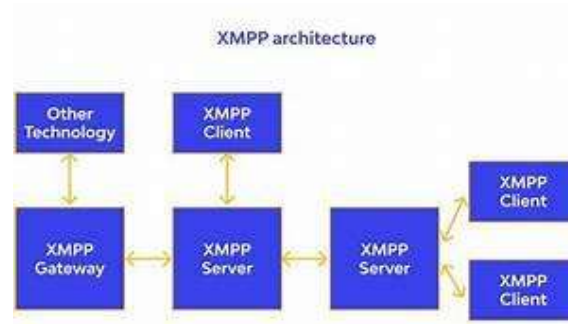
7. Telecommunications

- Telecom companies use AMQP for billing systems, service orchestration, and subscriber event handling.
- Enables fault-tolerant, high-throughput messaging across distributed networks.

5. XMPP (Extensible Messaging and Presence Protocol):

XMPP is an **XML-based real-time messaging protocol** originally designed for instant messaging and presence. In IoT, XMPP can be used for device-to-device communication or for notifying presence/status of devices. It runs over TCP and supports long-lived sessions.

- **Features:** Uses an open federated architecture with unique IDs (JIDs). XMPP streams small XML "stanzas" between entities. It inherently supports *presence* (knowing if a device/client is online), and has extensions for publish/subscribe (XMPP PubSub) and remote service invocation. Because XMPP is bidirectional and persistent, it's suited to applications needing realtime chat-like interaction or status updates.
- **Example:** A smart home hub could use XMPP to track its devices: each device connects to the XMPP server and announces its presence. When a door sensor opens, it sends an XMPP message (a presence changes or custom stanza) to the hub, which immediately triggers an alarm or notification. Voice-assistant devices also sometimes use XMPP to stream events or alerts.



XMPP follows a **client–server–gateway model**, ensuring efficient real-time communication.

1. XMPP Client

- These are end-user applications (like chat apps, IoT devices, or presence trackers).
- Clients connect to an XMPP server using XML-based messages called *stanzas*.
- Functions:
 - Send and receive messages.
 - Maintain presence information (online, offline, busy, etc.).
 - Subscribe to services like group chat, file sharing, or IoT data exchange.

2. XMPP Server

- Acts as the **central hub** for handling communication.
- Responsibilities:
 - **Route messages** between clients connected to the same server.
 - **Inter-server communication**: connect with other XMPP servers to forward messages to external clients.
 - **Authentication & authorization**: validate connected clients.
 - **Presence management**: keeps track of online/offline states.

3. Inter-server Communication

- XMPP servers can directly communicate with each other.
- Example: A client on **Server A** can chat with a client on **Server B** through server-to-server links.
- This creates a **federated network** (like email), meaning no single point of control.

4. XMPP Gateway

- A special component that connects XMPP to **other technologies/protocols** (e.g., SMS, email, or legacy messaging systems).
- It translates XMPP stanzas into another protocol's format and vice versa.
- Example: XMPP-to-SMS gateway allows sending instant messages as text messages.

5. Other Technology

- Represents external systems connected through gateways (e.g., traditional email, IoT platforms, or proprietary chat apps).

Applications:

1. Smart Homes & Automation

- Device presence management (knowing which devices are online/offline).

- Example: Door sensors, smart bulbs, and alarms notifying the hub instantly.

2. Healthcare & Wearables

- Real-time sharing of patient vitals (heart rate, blood pressure, oxygen levels).
- Enables remote monitoring and emergency alerts.

3. Industrial IoT (IIoT)

- Factory machines sending live status updates and fault notifications.
- Monitoring and controlling production lines remotely.

4. Smart Cities

- Managing streetlights, traffic signals, and energy meters.
- Sending real-time alerts for accidents, congestion, or public safety.

5. Collaboration & Messaging Platforms

- Used in chat systems like Google Talk (earlier), WhatsApp (initial versions), Slack gateways.
- Supports group chat, file sharing, and presence info.

6. Real-time Notifications

- Event-driven IoT applications such as fire alarms, motion detection, and intrusion alerts.
- Voice assistants (e.g., Alexa-type devices) streaming alerts via XMPP.

7. Cross-Protocol Communication via Gateways

- Bridging IoT devices with SMS, Email, or legacy messaging systems.
- Example: XMPP-to-SMS gateways for remote alerts.

8. Education & Research IoT Platforms

- Universities and labs use XMPP for testbeds involving **sensor-to-sensor messaging**.

LOGICAL DESIGN OF IOT

Logical design of an IoT system refers to an abstract representation of the entities and processes without going into the low-level specifics of the implementation. In this section we describe the functional blocks of an IoT system and the communication APIs that are used for the examples in this book.

1. IoT Functional Blocks

An IoT system comprises of a number of functional blocks that provide the system the capabilities for identification, sensing, actuation, communication, and management as shown in Figure. These functional blocks are described as follows:

- **Device:** An IoT system comprises of devices that provide sensing, actuation, monitoring and control functions.
- **Communication:** The communication block handles the communication for the IoT system.
- **Services:** An IoT system uses various types of IT services such as services for device monitoring, device control services, data publishing services and services for device discovery.
- **Management:** Management functional block provides various functions to govern the IoT system.
- **Security:** Security functional block secures the IoT system and by providing functions such as authentication, authorization, message and content integrity, and data security.
- **Application:** IoT applications provide an interface that the users can use to control and monitor various aspects of the IoT system. Applications also allow users to view the system status and view or analyze the processed data.

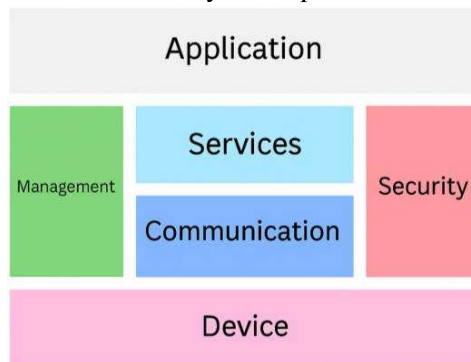
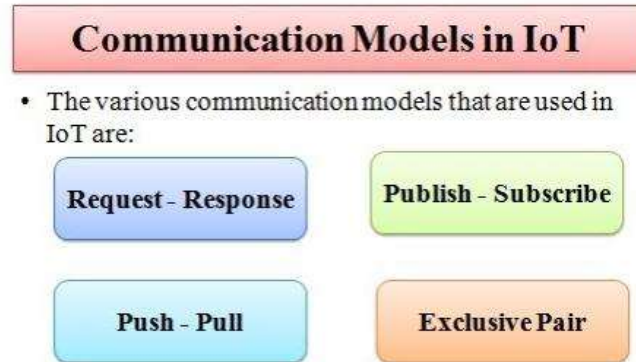


Figure 1.1: Functional Blocks of IOT

2. IoT Communication Models

Communication models in the Internet of Things (IoT) define how devices, services, and applications exchange information. These models outline the interaction patterns between producers (devices or sensors) and consumers (applications or services) of data, considering factors like timeliness, scalability, and complexity. Depending on the

application requirements, IoT systems may adopt synchronous request–response patterns, asynchronous publish–subscribe mechanisms, queue-based push–pull methods, or persistent exclusive pair connections.



2.1 Request-Response: Request-Response is a communication model in which the client sends requests to the server and the server responds to the requests. When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response, and then sends the response to the client. Request-Response model is a stateless communication model, and each request-response pair is independent of others. Below Figure shows the client-server interactions in the request-response model.

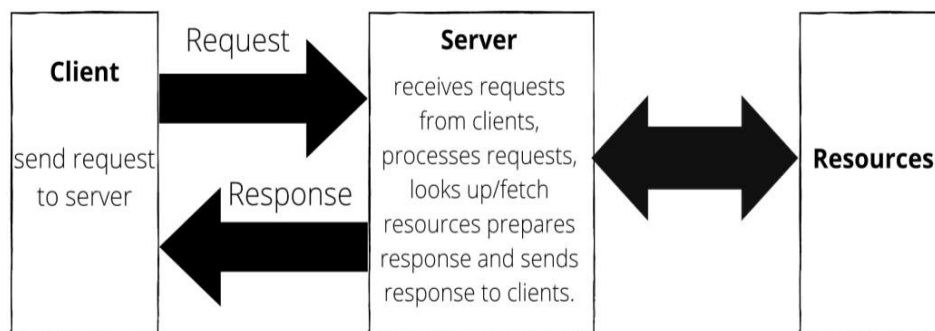


Figure 2.1: Request-response Communication Model

2.2 Publish-Subscribe: Publish-Subscribe is a communication model that involves publishers, brokers and consumers. Publishers are the source of data. Publishers send the data to the topics which are managed by the broker. Publishers are not aware of the consumers. Consumers subscribe to the topics which are managed by the broker. When the broker receives data for a topic from the publisher, it sends the data to all the

subscribed consumers. Below Figure shows the publisher-broker-consumer interactions in the publish-subscribe model.

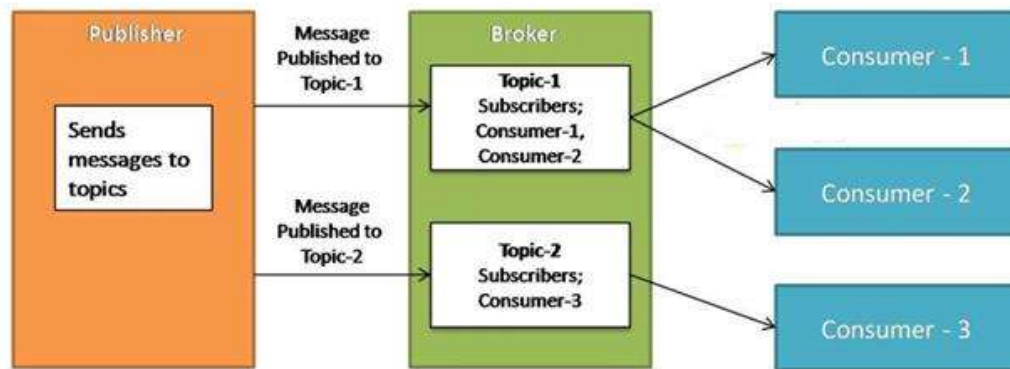


Figure 2.2: Publish-Subscribe Communication Model

2.3 Push-Pull: Push-Pull is a communication model in which the data producers push the data to queues and the consumers pull the data from the queues. Producers do not need to be aware of the consumers. Queues help in decoupling the messaging between the producers and consumers. Queues also act as a buffer which helps in situations when there is a mismatch between the rate at which the producers push data and the rate at which the consumers pull data. Below Figure shows the publisher-queue-consumer interactions in the push-pull model.

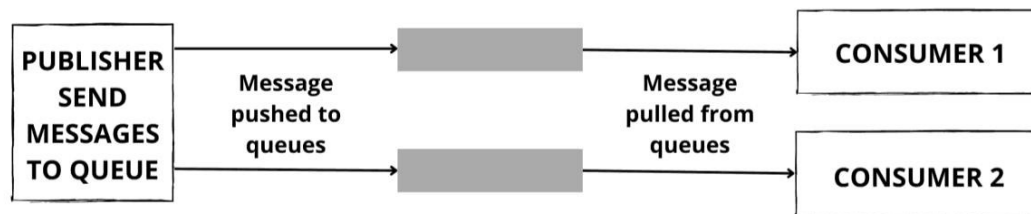


Figure 2.3: Push-Pull Communication Model

2.4 Exclusive Pair: Exclusive Pair is a bi-directional, fully duplex communication model that uses a persistent connection between the client and server. Once the connection is setup it remains open until the client sends a request to close the connection. Client and server can send messages to each other after connection setup. Exclusive pair is a stateful communication model and the server is aware of all the open connections. Below Figure shows the client-server interactions in the exclusive pair model.



Figure 2.4: Exclusive Pair Communication Model

IOT COMMUNICATION API's

IoT communication APIs define how IoT applications and devices interact with each other, cloud platforms, or user interfaces. They serve as standardized mechanisms for data exchange, enabling interoperability between heterogeneous systems.

Two widely used communication API models in IoT are:

1. Rest-based Communication APIs

Representational State Transfer (REST) is a set of architectural principles for designing web services and APIs that focus on system resources and the way resource states are addressed and transferred. REST APIs follow the **request-response** communication model.

The REST architectural constraints apply to the **components**, **connectors**, and **data elements** within a distributed hypermedia system.

REST Architectural Constraints

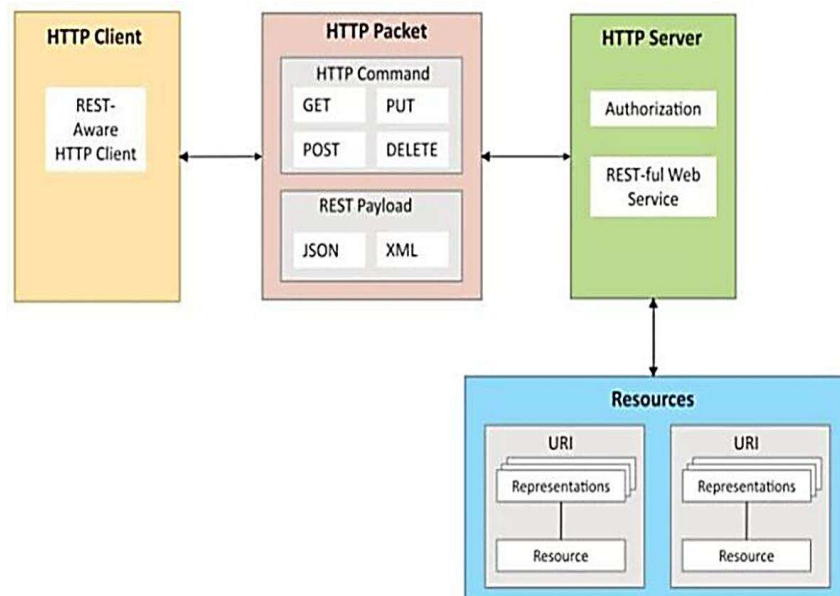


Figure 1: Communication with REST APIs

1. Client–Server

- Separation of concerns is the core principle.
- The client is not concerned with data storage (server's role), and the server is not concerned with the user interface (client's role).
- Enables independent development and updates of client and server components.

2. Stateless

- Each request from client to server must contain all required information to process the request.
- The server does not store session state; the state is entirely maintained by the client.

3. Cache-able

- Responses must be explicitly or implicitly marked as cache-able or non-cache-able.
- Caching improves efficiency and scalability by reusing data for future equivalent requests.

4. Layered System

- Components can only see and interact with the immediate layer they communicate with.
- The client cannot detect whether it is connected to the end server or an intermediary.
- Improves scalability by enabling intermediaries (e.g., proxies, gateways) to handle requests.

5. Uniform Interface

- Communication between client and server must follow a uniform approach.

- Resources are identified via **URIs** and are separate from their representations.
- A resource representation contains all necessary information for updating or deleting it (if permissions allow).
- Each message is self-descriptive.

6. Code on Demand (*Optional*)

- Servers can provide executable code (e.g., scripts) for clients to execute locally.

REST Request–Response Model

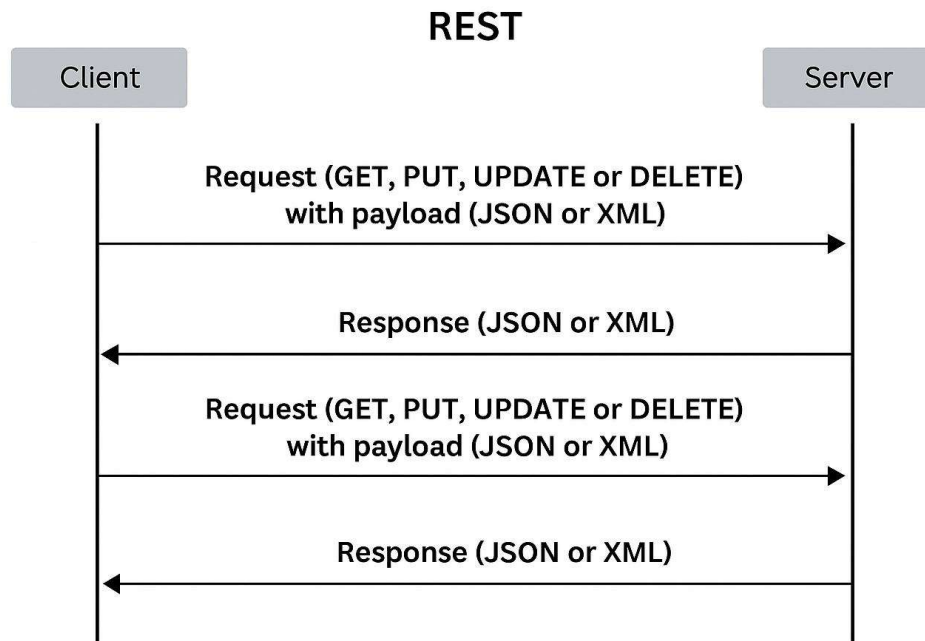


Figure 2: Request–Response Model used by REST

REST (Representational State Transfer) is an HTTP-based request–response communication model for client-server interaction.

- **Connection Behavior:** Does not maintain a persistent connection; each request is independent.
- **HTTP Methods:** Uses standard methods such as GET, PUT, UPDATE, and DELETE.

- **Data Formats:** Requests and responses typically use JSON or XML for data representation.

The **request–response cycle** repeats for each client–server interaction:

1. **Client Request:** The client sends a request (GET, PUT, UPDATE, or DELETE) with an optional payload (JSON/XML).
2. **Server Response:** The server returns a response containing data or a status message in JSON/XML.
3. The cycle is repeated for subsequent requests.

REST is well-suited for IoT scenarios that require **simple, scalable, and loosely coupled communication** between distributed devices and services, but it may introduce higher latency compared to persistent connections like WebSockets, especially in real-time applications.

HTTP Request Methods in REST

HTTP Method	Resource Type	Action	Example
GET	Collection URI	List all the resources in a collection	http://example.com/api/tasks/ (list all tasks)
GET	Element URI	Get information about a resource	http://example.com/api/tasks/1/ (get information on task-1)
POST	Collection URI	Create a new resource	http://example.com/api/tasks/ (create a new task from data provided in the request)
POST	Element URI	Generally not used	
PUT	Collection URI	Replace the entire collection with another collection	http://example.com/api/tasks/ (replace entire collection with data provided in the request)
PUT	Element URI	Update a resource	http://example.com/api/tasks/1/ (update task-1 with data provided in the request)
DELETE	Collection URI	Delete the entire collection	http://example.com/api/tasks/ (delete all tasks)
DELETE	Element URI	Delete a resource	http://example.com/api/tasks/1/ (delete task-1)

Table 1: HTTP Request Methods and Actions

- Base URI: Example – <http://example.com/api/tasks/>

- Popular Media Types: JSON (most common) and XML.
- The IP for Smart Objects Alliance (IPSO Alliance) has published an Application Framework defining RESTful design for IP smart object systems.

2. WebSocket-based Communication APIs

WebSocket APIs allow bi-directional, full duplex communication between clients and servers. WebSocket APIs follow the exclusive pair communication model described in previous section and as shown in Figure. Unlike request-response APIs such as REST, the WebSocket APIs allow full duplex communication and do not require a new connection to be setup for each message to be sent.

WebSocket communication begins with a connection setup request sent by the client to the server. This request (called a WebSocket handshake) is sent over HTTP and the server interprets it as an upgrade request. If the server supports WebSocket protocol, the server responds to the WebSocket handshake response. After the connection is setup, the client and server can send data/messages to each other in full-duplex mode. WebSocket APIs reduce the network traffic and latency as there is no overhead for connection setup and termination requests for each message. WebSocket is suitable for IOT applications that have low latency or high throughput requirements.

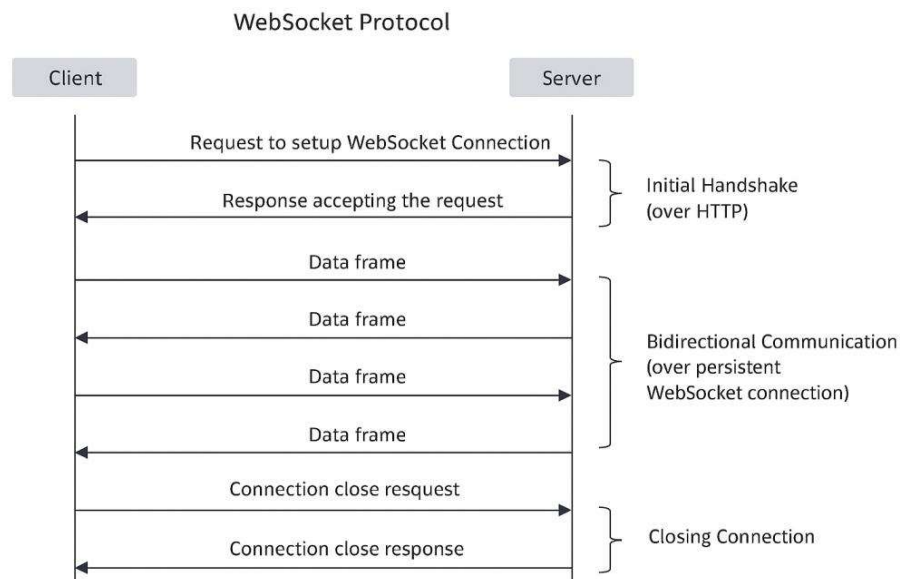


Figure 1.2 Exclusive Pair model used by WebSocket API

IOT ENABLING TECHNOLOGIES

IoT is enabled by several technologies including wireless sensor networks, cloud computing, big data analytics, embedded systems, security protocols and architectures, communication protocols, web services, mobile Internet, and semantic search engines. This section provides an overview of some of these technologies which play a key-role in IoT.

9. Wireless Sensor Networks

A Wireless Sensor Network (WSN) comprises of distributed devices with sensors which are used to monitor the environmental and physical conditions. A WSN consist of a number of end-nodes and routers and a coordinator. End nodes have several sensors attached to them. End nodes can also act as routers. Routers are responsible for routing the data packets from end-nodes to the coordinator. The coordinator collects the data from all the nodes. Coordinator also acts as a gateway that connects the WSN to the Internet. Some examples of WSNs used in IoT systems are described as follows:

- Weather monitoring systems use WSNs in which the nodes collect temperature, humidity and other data, which is aggregated and analyzed.
- Indoor air quality monitoring systems use WSNs to collect data on the indoor air quality and concentration of various gases.
- Soil moisture monitoring systems use WSNs to monitor soil moisture at various locations.
- Surveillance systems use WSNs for collecting surveillance data (such as motion detection data)
- Smart grids use WSNs for monitoring the grid at various points.
- Structural health monitoring systems use WSNs to monitor the health of structures (buildings, bridges) by collecting vibration data from sensor nodes deployed at various points in the structure.

WSNs are enabled by wireless communication protocols such as IEEE 802.15.4. ZigBee is one of the most popular wireless technologies used by WSNs. ZigBee specifications are based on IEEE 802.15.4. ZigBee operates at 2.4 GHz frequency and offers data rates upto 250 KB/s and range from 10 to 100 meters depending on the power output and environmental conditions. The power of WSNs lies in their ability to deploy large number of low-cost and low-power sensing nodes for continuous monitoring of environmental and physical conditions. WSNs are self-organizing networks. Since WSNs have large number of nodes, manual configuration for each node is not possible. The self-organizing capability of WSN makes the network robust. In the

event of failure of some nodes or addition of new nodes to the network, the network can reconfigure itself.

10. Cloud Computing

Cloud computing is a transformative computing paradigm that involves delivering applications and services over the Internet. Cloud computing involves provisioning of computing, networking and storage resources on demand and providing these resources as metered services to the users, in a "pay as you go" model. Cloud computing resources can be provisioned on-demand by the users, without requiring interactions with the cloud service provider. The process of provisioning resources is automated. Cloud computing resources can be accessed over the network using standard access mechanisms that provide platform-independent access through the use of heterogeneous client platforms such as workstations, laptops, tablets and smart-phones.

The computing and storage resources provided by cloud service providers are pooled to serve multiple users using multi-tenancy. Multi-tenant aspects of the cloud allow multiple users to be served by the same physical hardware. Users are assigned virtual resources that run on top of the physical resources.

Cloud computing services are offered to users in different forms (see the authors' companion book on Cloud Computing, for instance):

- **Infrastructure-as-a-Service (IaaS):** IaaS provides the users the ability to provision computing and storage resources. These resources are provided to the users as virtual machine instances and virtual storage. Users can start, stop, configure and manage the virtual machine instances and virtual storage. Users can deploy operating systems and applications of their choice on the virtual resources provisioned in the cloud. The cloud service provider manages the underlying infrastructure. Virtual resources provisioned by the users are billed based on a pay-per-use paradigm.
- **Platform-as-a-Service (PaaS):** PaaS provides the users the ability to develop and deploy application in the cloud using the development tools, application programming interfaces (APIs), software libraries and services provided by the cloud service provider. The cloud service provider manages the underlying cloud infrastructure including servers, network, operating systems and storage. The users, themselves, are responsible for developing, deploying, configuring and managing applications on the cloud infrastructure.

- **Software-as-a-Service (SaaS):** SaaS provides the users a complete software application or the user interface to the application itself. The cloud service provider manages the underlying cloud infrastructure including servers, network, operating systems, storage and application software, and the user is unaware of the underlying architecture of the cloud. Applications are provided to the user through a thin client interface (e.g., a browser). SaaS applications are platform independent and can be accessed from various client devices such as workstations, laptop, tablets and smart-phones, running different operating systems. Since the cloud service provider manages both the application and data, the users are able to access the applications from anywhere.

11. Big Data Analytics

Big data is defined as collections of data sets whose volume, velocity (in terms of its temporal variation), or variety, is so large that it is difficult to store, manage, process and analyze the data using traditional databases and data processing tools. Big data analytics involves several steps starting from data cleansing, data munging (or wrangling), data processing and visualization. Some examples of big data generated by IoT systems are described as follows:

- Sensor data generated by IoT systems such as weather monitoring stations.
- Machine sensor data collected from sensors embedded in industrial and energy systems for monitoring their health and detecting failures.
- Health and fitness data generated by IoT devices such as wearable fitness bands.
- Data generated by IoT systems for location and tracking of vehicles.
- Data generated by retail inventory monitoring systems.

The underlying characteristics of big data include:

- **Volume:** Though there is no fixed threshold for the volume of data to be considered as big data, however, typically, the term big data is used for massive scale data that is difficult to store, manage and process using traditional databases and data processing architectures. The volumes of data generated by modern IT, industrial, and health-care systems, for example, is growing exponentially driven by the lowering costs of data storage and processing architectures and the need to extract valuable insights from the data to improve business processes, efficiency and service to consumers.
- **Velocity:** Velocity is another important characteristic of big data and the primary reason for exponential growth of data. Velocity of data refers to how fast the data is generated

and how frequently it varies. Modern IT, industrial and other systems are generating data at increasingly higher speeds.

- **Variety:** Variety refers to the forms of the data. Big data comes in different forms such as structured or unstructured data, including text data, image, audio, video and sensor data.

12. Communication Protocols

Communication protocols act as the **foundation of IoT (Internet of Things) systems**, ensuring seamless **data transfer, connectivity, and interaction** between devices, networks, and applications. Without these protocols, IoT devices would not be able to understand or reliably exchange information with one another.

Protocols establish a set of **rules and standards** that govern:

- How devices are addressed on the network.
- How data is formatted, encoded, and transmitted.
- How packets are routed from a **source device** to the **destination device or application**.

Role of Communication Protocols in IoT

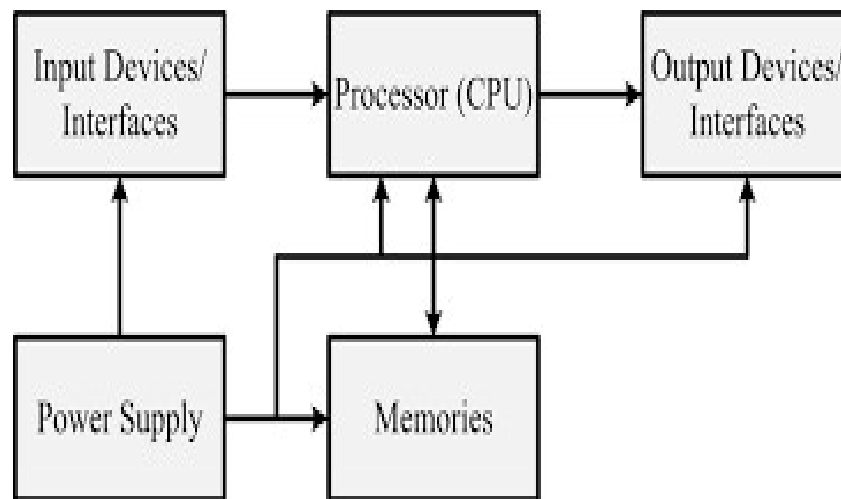
1. **Data Exchange** – Protocols define the structure (message formats) and meaning of data so devices from different vendors can communicate effectively.
2. **Data Encoding** – They specify how raw data is represented (binary, text, or XML/JSON formats).
3. **Addressing** – Each device in the network requires a unique identifier (like an IP address or device ID) to be located and contacted.
4. **Routing** – Protocols manage the path that data packets take across the network to reach the correct destination.

13. Embedded Systems

An Embedded System is a computer system that has computer hardware and software embedded to perform specific tasks. In contrast to general purpose computers or personal computers (PCs) which can perform various types of tasks, embedded systems are designed to perform a specific set of tasks. Key components of an embedded system include, microprocessor or microcontroller, memory (RAM, ROM, cache), networking units (Ethernet, WiFi adapters), input/output units (display, keyboard, etc.) and storage (such as flash memory).

Some embedded systems have specialized processors such as digital signal processors (DSPs), graphics processors and application specific processors. Embedded systems run embedded operating systems such as real-time operating systems (RTOS). Embedded systems range from low-cost miniaturized devices such as digital watches to devices such as digital cameras, point of sale terminals, vending machines, appliances (such as washing machines), etc. In the next chapter we describe how such devices form an integral part of IoT systems.

Block Diagram of Embedded System



The block diagram consists of:

1. Input Devices / Interfaces (Sensors)

- Collect information from the environment (temperature, humidity, motion, light, pressure, gas levels, etc.).
- Convert physical quantities into electrical/digital signals.
- Example: DHT11 (temperature & humidity), PIR sensor (motion detection).

2. Processor (CPU / Microcontroller / Microprocessor)

- Acts as the brain of the embedded system.
- Executes instructions stored in memory and processes input signals.
- Makes decisions in real-time and controls actuators or communication modules.

- Examples: ARM Cortex-M series, ESP32, Raspberry Pi SoC.

3. Memory

- Stores both the program (firmware) and data.
- ROM / Flash: Holds permanent code and system instructions.
- RAM: Provides temporary storage for data during execution.
- In IoT devices, memory requirements are small but optimized for speed and efficiency.

4. Output Devices / Interfaces (Actuators)

- Receive processed signals from the processor and perform required actions.
- Convert digital outputs into real-world actions.
- Examples: motors (for movement), relays (for switching appliances), LEDs/buzzers (for alerts).

5. Power Supply

- Supplies stable power to all components.
- IoT embedded systems are often battery-operated (Li-ion, coin-cell batteries) or powered by renewable energy (solar panels).
- Efficiency is crucial to support long-term operation of IoT devices.

Working of Embedded Systems in IoT

The working of an embedded system in an IoT environment follows these steps:

1. **Sensing:** Sensors capture data from the physical world.
 - Example: A temperature sensor records room temperature.
2. **Processing:** The processor analyzes the sensed data, applies algorithms, and makes logical decisions.
 - Example: If the temperature $> 30^{\circ}\text{C}$, turn ON the cooling system.
3. **Storage:** Data is temporarily stored in RAM or permanently logged in flash memory.
4. **Action:** Actuators execute the required operation.
 - Example: A motor turns ON the fan, or an alarm is triggered.

5. **Communication:** Data is transmitted via communication modules (Wi-Fi, Zigbee, LoRa, Bluetooth, 5G) to cloud servers for remote access.
 - Example: The room temperature is sent to a mobile app through MQTT or CoAP.